Task 1: Foundation – Successfully Create and Terminate an EMR Cluster

Objective

The goal of this task is to set up an Amazon EMR cluster for the first time, ensuring that the cluster is properly created, becomes active, and can be terminated to avoid unnecessary costs. This exercise helps in overcoming the initial hurdles of working with EMR.

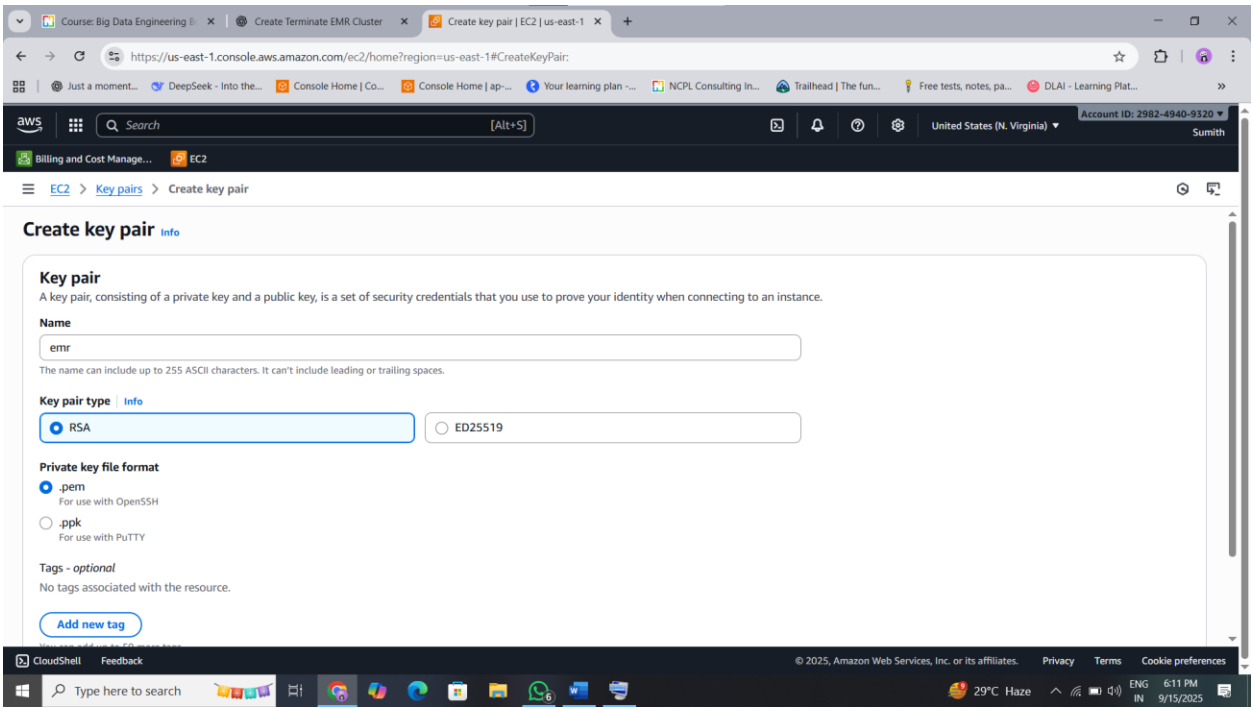
Theory

- **Amazon EMR (Elastic MapReduce):** A managed cluster platform on AWS that simplifies running big data frameworks such as Apache Spark, Hadoop, and Hive.
- **Key Pair:** A cryptographic key that provides secure access to EC2 instances. You need to create one before launching the cluster.
- **Cluster Configuration:** Choosing the right EMR release, applications, and instance types ensures the cluster matches learning or workload needs.
- **Auto-Termination:** A cost-saving feature where the cluster automatically shuts down after being idle for a set time. This is critical during practice to avoid high AWS bills.
- **Termination:** After the experiment, the cluster and its underlying EC2 instances must be terminated manually to release resources.

Steps

Step 1: Key Pair Creation

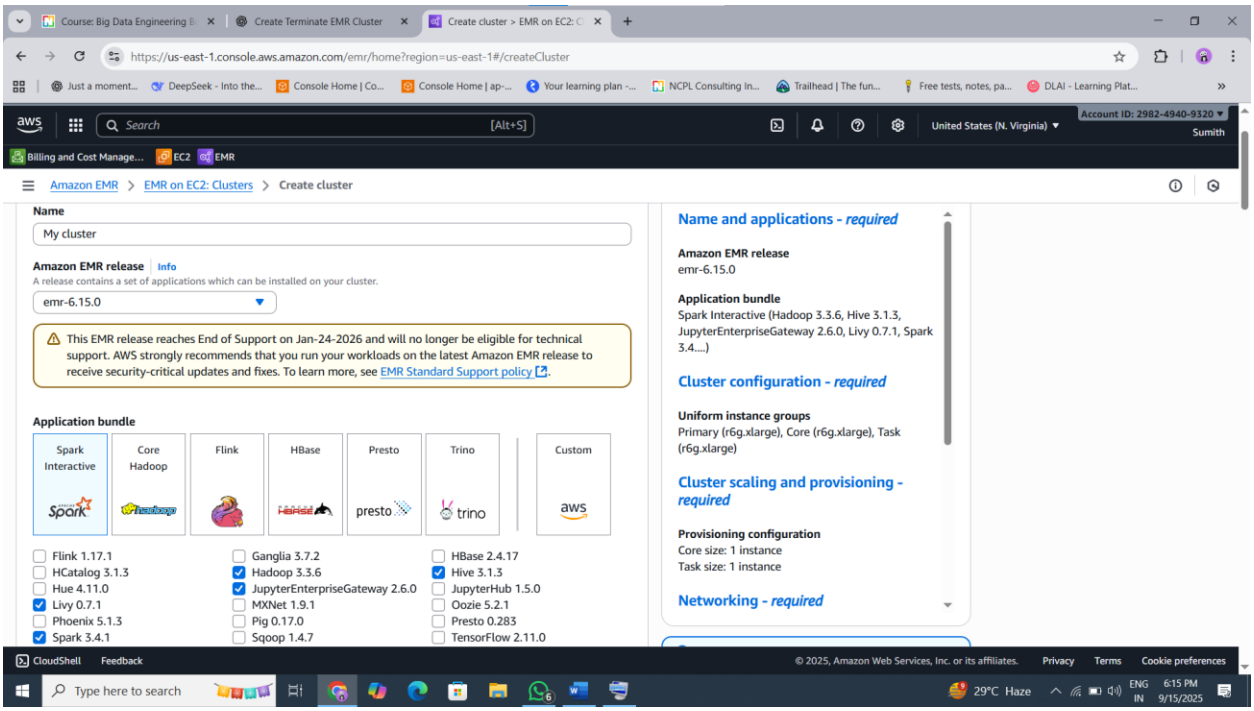
1. Log in to the **AWS Management Console**.
2. Navigate to **EC2 → Key Pairs**.
3. Click **Create key pair**.
    - Name: emr-keypair.
    - Region: **us-east-1 (N. Virginia)**.
    - File format: .pem (for Linux/macOS) or .ppk (for Windows).



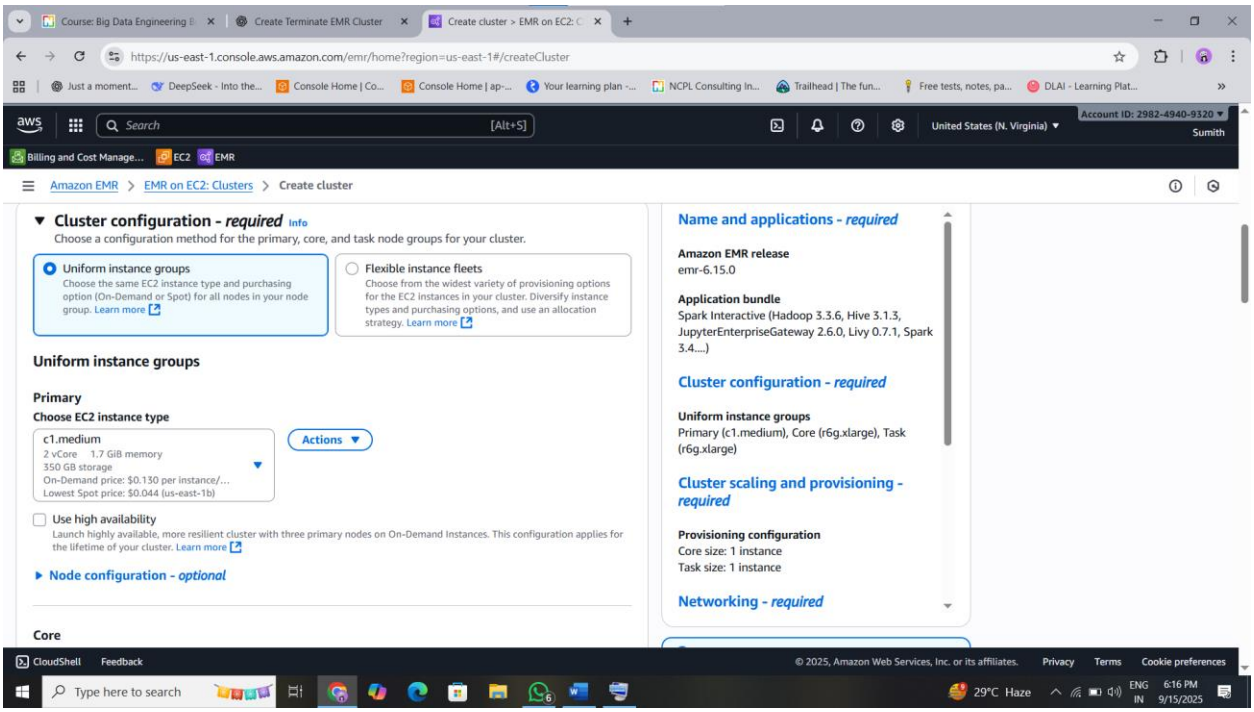
4. Download the key file and store it securely.
    - For Linux/macOS, run:
    - chmod 400 emr-keypair.pem

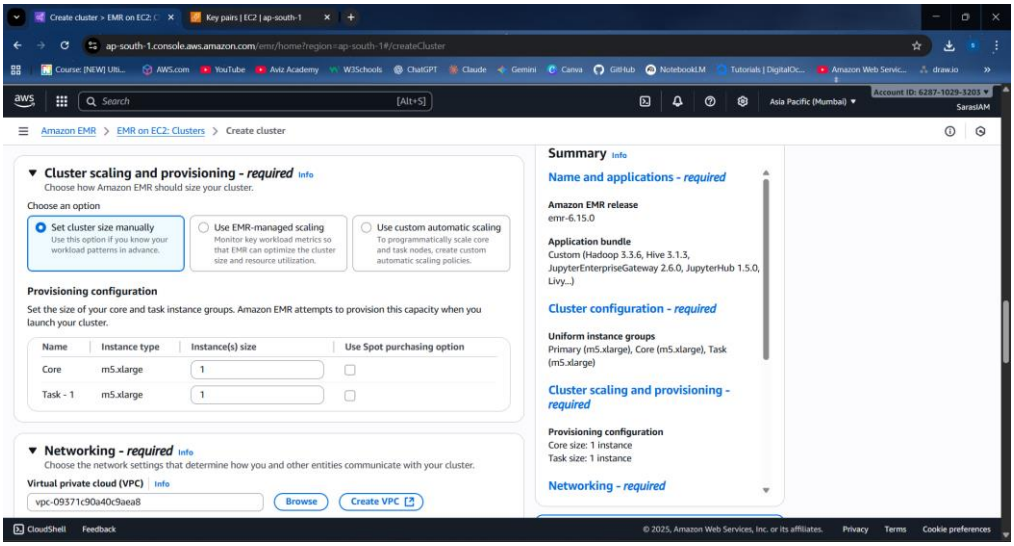
Step 2: Cluster Creation

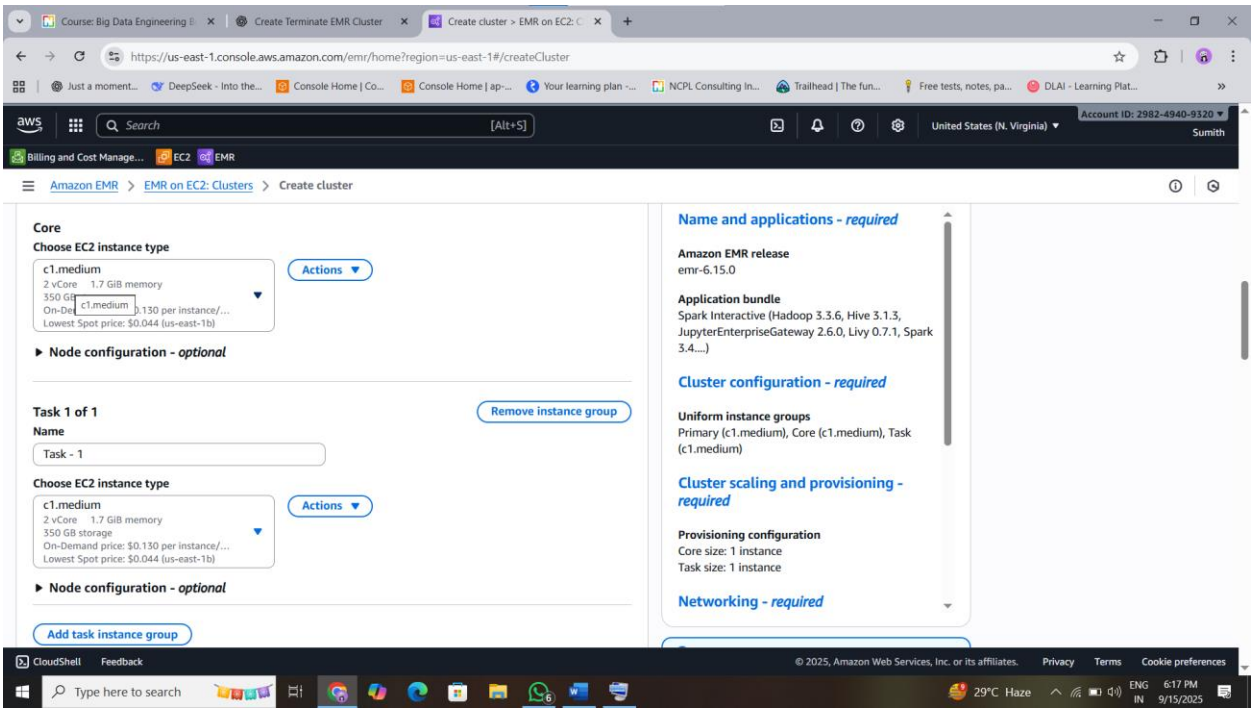
1. Go to **Amazon EMR → Create cluster → Advanced options**.
2. Configure the following:
    - **Release:** emr-6.15.0.
    - **Applications:** Spark, Hadoop, JupyterHub.



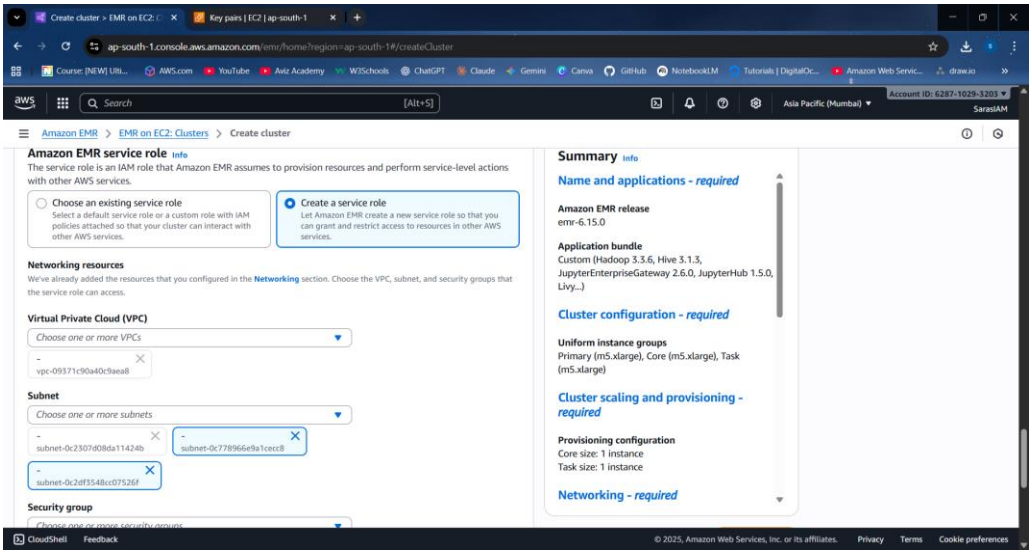
- **Hardware Configuration:**
  - Master: c1.medium.
  - Core: c1.medium (1 node for practice).



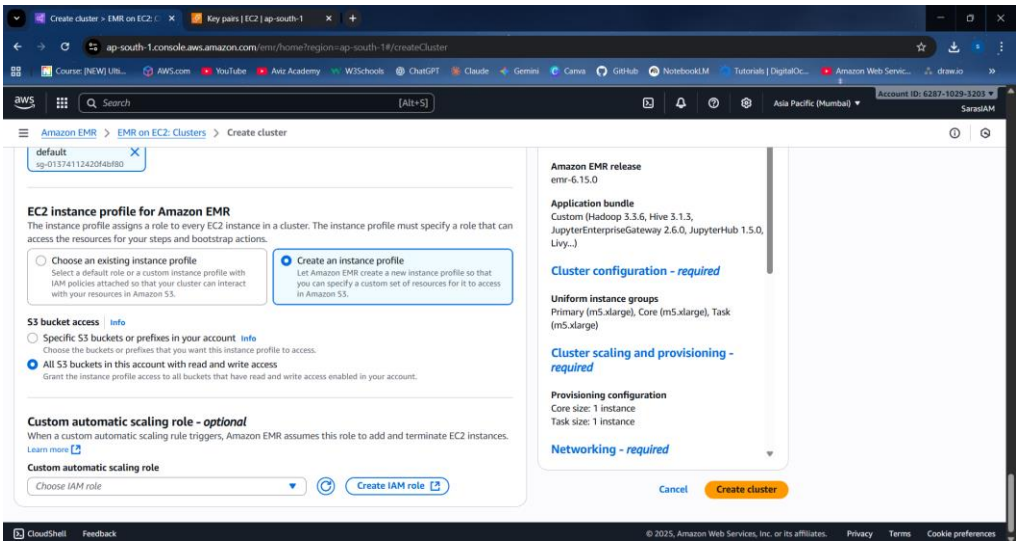




- **EC2 Key Pair:** Select emr-keypair.
- **Auto-Termination:** Enable and set to **60 minutes**.

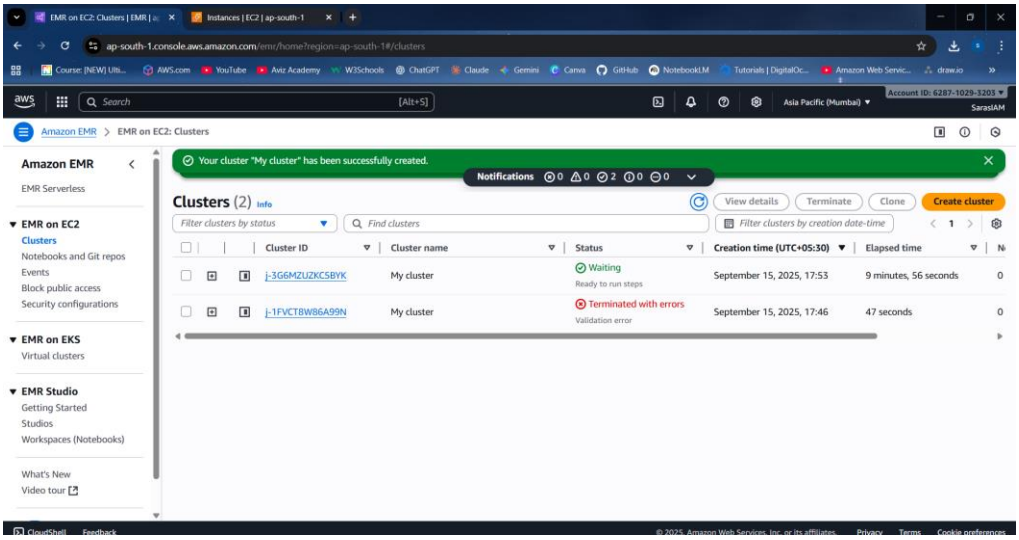


3. Click **Create cluster**.

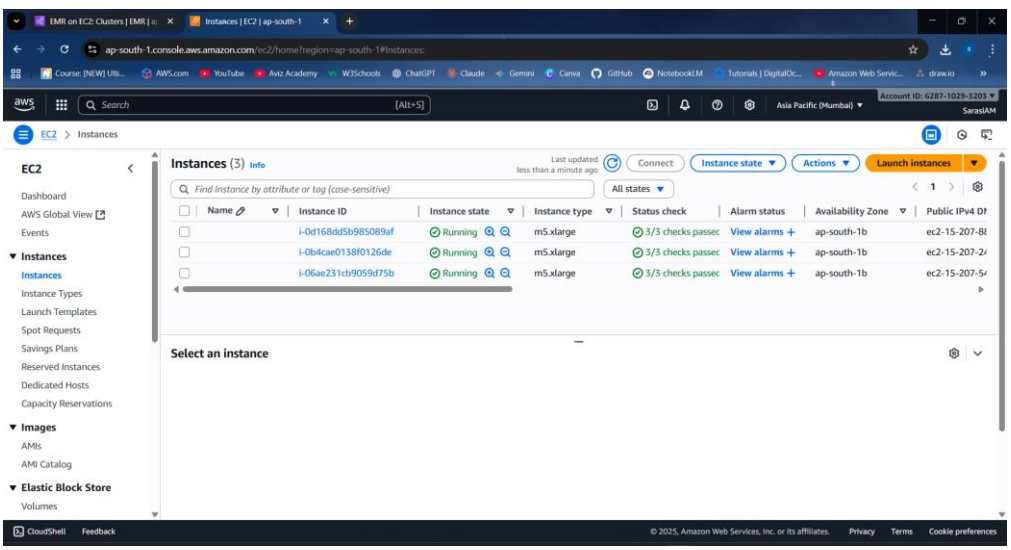


Step 3: Verification

1. In the EMR console, monitor the cluster status.
    - It will progress from **Starting → Bootstrapping → Waiting**.

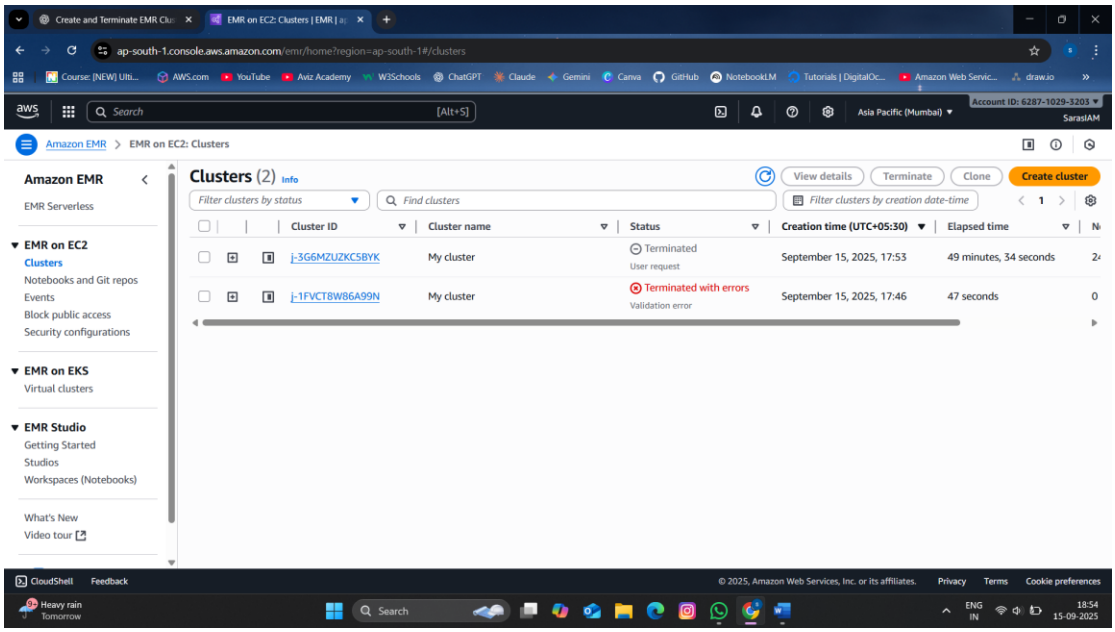


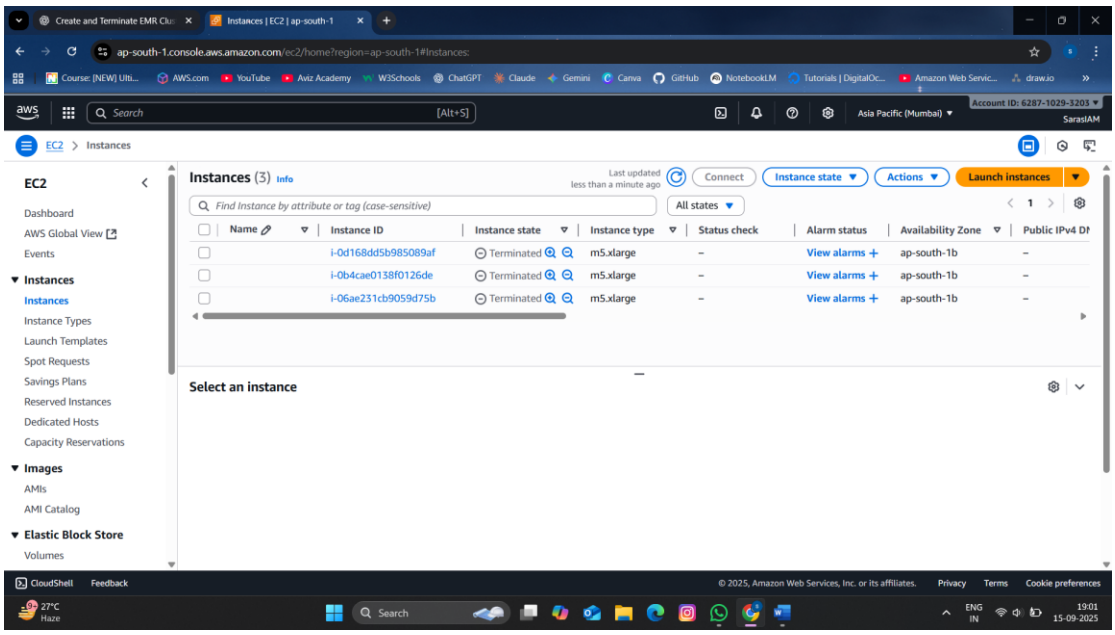
2. Ensure the cluster reaches the **Waiting** state, meaning it is ready.
3. (Optional) Check in **EC2 Console** → verify that EMR created EC2 instances.



Step 4: Termination

1. Select the cluster in the EMR console.
2. Click **Terminate** and confirm.
3. Navigate to **EC2 → Instances** and verify that all cluster-related EC2 instances are terminated.





Result: You have successfully created, verified, and terminated an EMR cluster while ensuring cost control through auto-termination.

Task 2: Application – Run a Sample Spark Job using a Step

Objective

The goal of this task is to learn how to submit and run a Spark job on an EMR cluster using **Steps**. Steps allow you to define processing jobs (like Spark, Hive, or custom JARs) that execute on the cluster. Here, we use the **SparkPi** example program to test job execution.

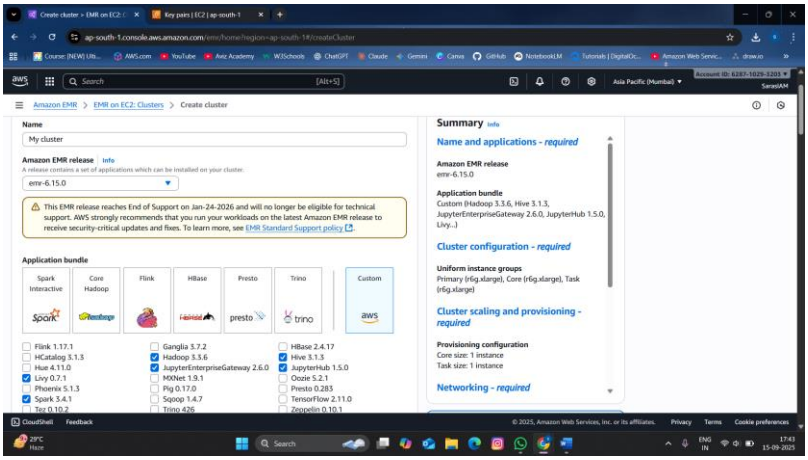
Theory

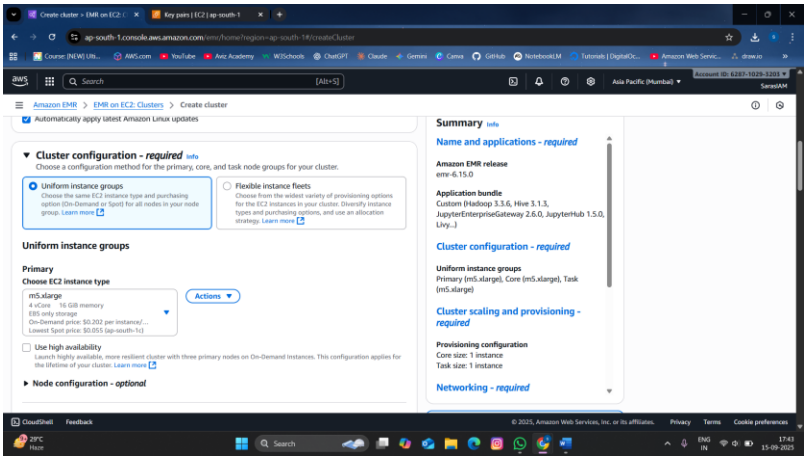
- **Steps in EMR:** A "Step" is a predefined unit of work (e.g., a Spark job or a Hadoop job) that EMR executes sequentially. Each step can be monitored for success or failure.
- **command-runner.jar:** A utility JAR available by default in EMR clusters that allows running commands such as spark-submit.
- **SparkPi Example:** A classic sample Spark program that calculates an approximate value of π (Pi) by performing parallel computations. It is lightweight and ideal for testing the Spark setup.
- **Auto-Termination:** Since the cluster has auto-termination enabled, it will shut down after the job finishes, ensuring no extra costs.

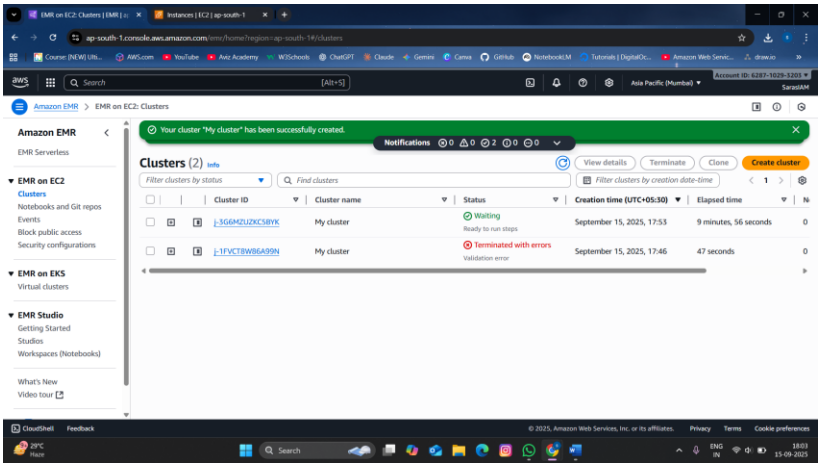
Steps

Step 1: Create Cluster

1. Go to the **Amazon EMR Console → Create cluster**.
2. Configure it the same way as in **Task 1**:
    - Release: emr-6.15.0.
    - Applications: Spark, Hadoop, JupyterHub.
    - Master: m5.xlarge.
    - Core: m5.xlarge (1 node).
    - EC2 Key Pair: Select the key created earlier (emr-keypair).
    - Auto-Termination: Enable (60 minutes).
3. Wait until the cluster reaches the **Waiting** state.

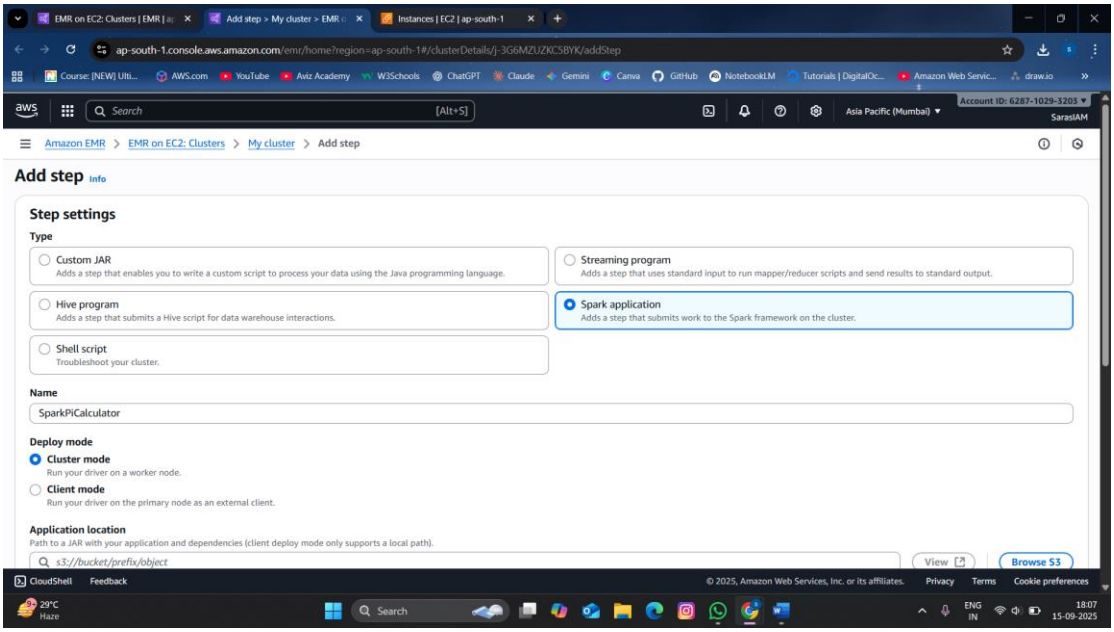

Amazon EMR > EMR on EC2: Clusters > Create cluster
Name
My cluster
Amazon EMR release
emr-6.15.0
This EMR release reaches End of Support on Jan-24-2026 and will no longer be eligible for technical support. AWS strongly recommends that you run your workloads on the latest Amazon EMR release to receive security-critical updates and fixes. To learn more, see EMR Standard Support policy
Application bundle
Spark Interactive
Core Hadoop
Flink
HBase
Presto
Trino
Custom
Flink 1.17.1
HCatalog 3.1.3
Hue 4.11.0
Livy 0.7.1
Phoenix 5.1.3
Spark 3.4.1
Tez 0.10.2
Ganglia 3.7.2
Hadoop 3.3.6
JupyterEnterpriseGateway 2.6.0
MXNet 1.9.1
Pig 0.17.0
Sqoop 1.4.7
Trino 426
HBase 2.4.17
Hive 3.1.3
JupyterHub 1.5.0
Oozie 5.2.1
Presto 0.283
TensorFlow 2.11.0
Zeppelin 0.10.1
Summary
Name and applications - required
Amazon EMR release
emr-6.15.0
Application bundle
Custom (Hadoop 3.3.6, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, JupyterHub 1.5.0, Livy...)
Cluster configuration - required
Uniform instance groups
Primary (r6g.xlarge), Core (r6g.xlarge), Task (r6g.xlarge)
Cluster scaling and provisioning - required
Provisioning configuration
Core size: 1 instance
Task size: 1 instance
Networking - required

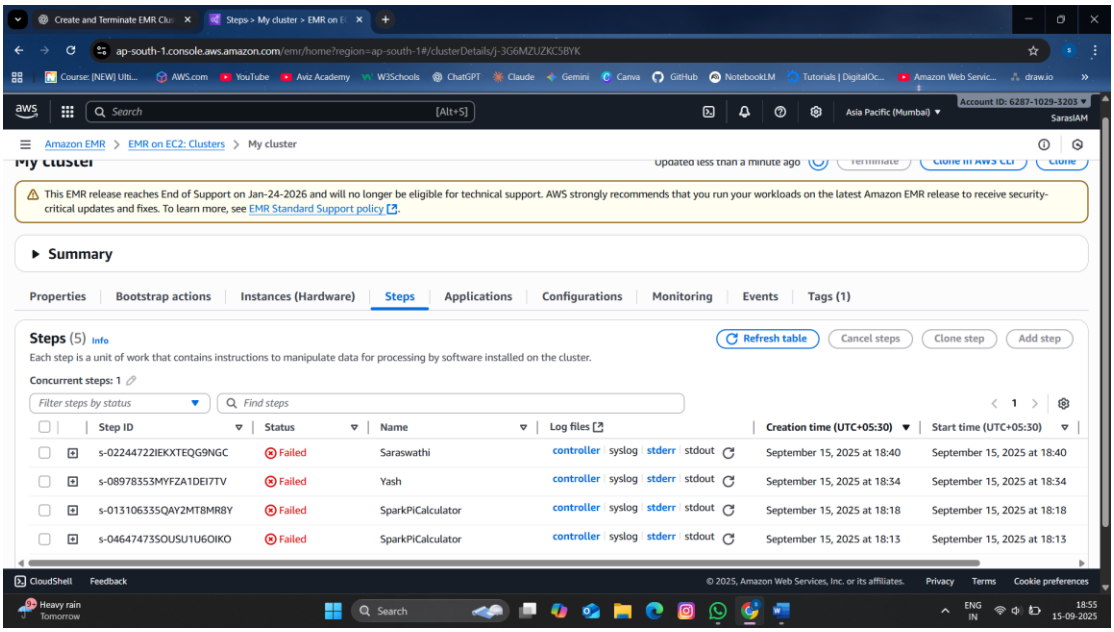

Amazon EMR > EMR on EC2: Clusters > Create cluster
Automatically apply latest Amazon Linux updates
Cluster configuration - required
Choose a configuration method for the primary, core, and task node groups for your cluster.
Uniform instance groups
Choose the same EC2 instance type and purchasing option (On-Demand or Spot) for all nodes in your node group.
Flexible instance fleets
Choose from the widest variety of provisioning options for the EC2 instances in your cluster. Diversify instance types and purchasing options, and use an allocation strategy.
Uniform instance groups
Primary
Choose EC2 instance type
m5.xlarge
Actions
Use high availability
Node configuration - optional
Summary
Name and applications - required
Amazon EMR release
emr-6.15.0
Application bundle
Custom (Hadoop 3.3.6, Hive 3.1.3, JupyterEnterpriseGateway 2.6.0, JupyterHub 1.5.0, Livy...)
Cluster configuration - required
Uniform instance groups
Primary (m5.xlarge), Core (m5.xlarge), Task (m5.xlarge)
Cluster scaling and provisioning - required
Provisioning configuration
Core size: 1 instance
Task size: 1 instance
Networking - required


Amazon EMR > EMR on EC2: Clusters
Amazon EMR
EMR Serverless
EMR on EC2
Clusters
Notebooks and Git repos
Events
Block public access
Security configurations
EMR on EKS
Virtual clusters
EMR Studio
Getting Started
Studios
Workspaces (Notebooks)
What's New
Video tour
Your cluster "My cluster" has been successfully created.
Notifications
Clusters (2)
View details
Terminate
Clone
Create cluster
Filter clusters by status
Find clusters
Filter clusters by creation date-time
Cluster ID
Cluster name
Status
Creation time (UTC+05:30)
Elapsed time
j-3G6MZUZKC5BYK
My cluster
Waiting
Ready to run steps
September 15, 2025, 17:53
9 minutes, 56 seconds
j-1FVCTBW86A99N
My cluster
Terminated with errors
Validation error
September 15, 2025, 17:46
47 seconds

Step 2: Add a Spark Step

1. Select the cluster in the EMR console.
2. Go to the **Steps** tab → Click **Add Step**.
3. Configure the step:
    - **Step type:** Spark application.
    - **Name:** Spark Pi Calculator.
    - **Application location:** command-runner.jar.
    - **Arguments:**
    - spark-submit --class org.apache.spark.examples.SparkPi /usr/lib/spark/examples/jars/spark-examples.jar 100
    - **Action on failure:** Continue.





Note : Every image is reference for working & got error on execution.

Step 3: Run and Monitor

1. Add the step and wait for execution.
2. Monitor the **Steps tab**: The status will progress from **Pending → Running → Completed**.
3. Once the step finishes, the cluster will automatically **terminate** after the idle timeout.

Result: You have successfully executed a Spark job on EMR using a **Step**. This proves the cluster is functional and can process distributed jobs.

Task 3: Exploration – Analyze the Cost and Service Quotas

Objective

The purpose of this task is to build awareness of **AWS costs** and **service limitations**. While EMR makes big data processing simple, the underlying infrastructure (EC2 instances, storage, networking) incurs costs and is subject to quotas. By monitoring expenses and checking service limits, you gain control over both budget and scalability.

Theory

- **AWS Cost Explorer:** A tool that provides visual reports of your AWS spending. You can filter by service (e.g., EMR, EC2) and time period to analyze costs. Costs for new activity may take 24–48 hours to appear.
- **EMR Costs:** Running EMR clusters incurs charges for the **EC2 instances**, **EBS volumes**, and sometimes **EMR service fees**. Since auto-termination was enabled in earlier tasks, costs should be minimal.
- **Service Quotas (Limits):** AWS imposes default limits (quotas) on resources to prevent accidental overuse. For example, EC2 has quotas for how many vCPUs you can run simultaneously.
- **vCPU Limit:** Determines how many virtual CPUs you can use across EC2 instances in a given family/region. This directly impacts how large of an EMR cluster you can create.

Steps

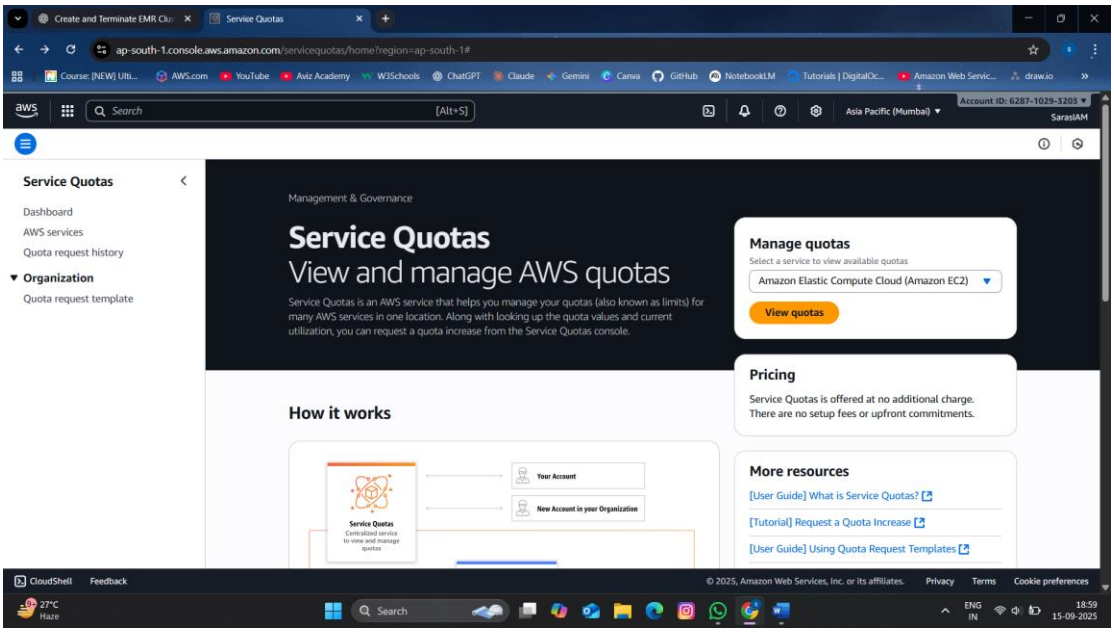
Step 1: Check Costs in Cost Explorer

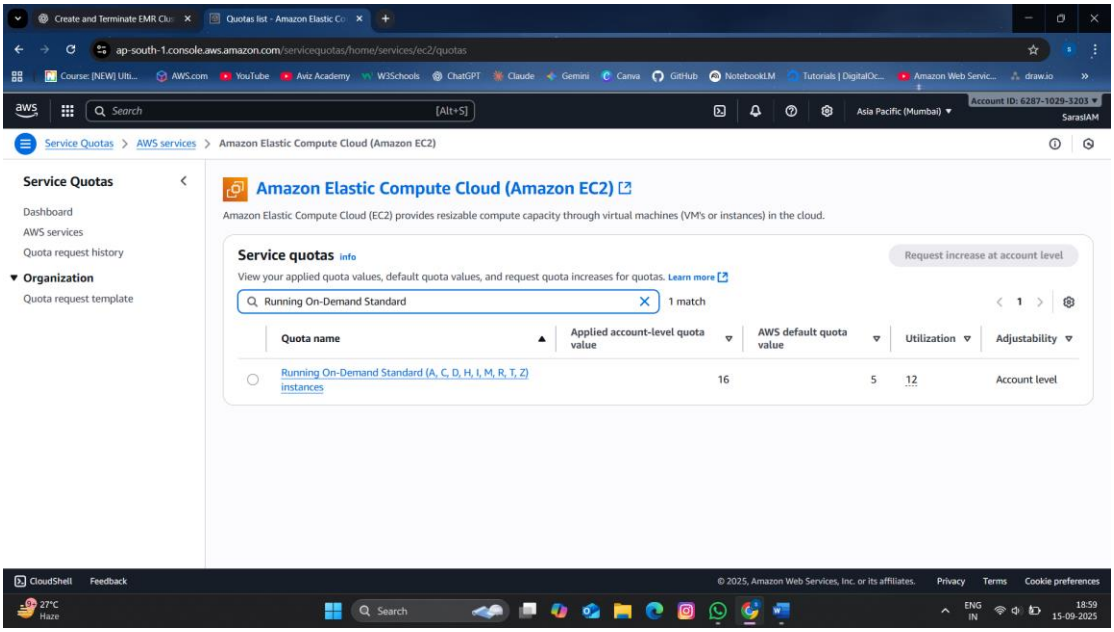
1. Open the **AWS Console → Cost Explorer** service.
2. Select **Current Month** as the time range.
3. Apply filters:
    - Service: **Amazon Elastic MapReduce (EMR)**
    - Service: **Amazon EC2**
4. Review the graph or table to estimate the cost incurred from the clusters you created in **Task 1** and **Task 2**.
    - Note: It may take **24–48 hours** before the charges show up.

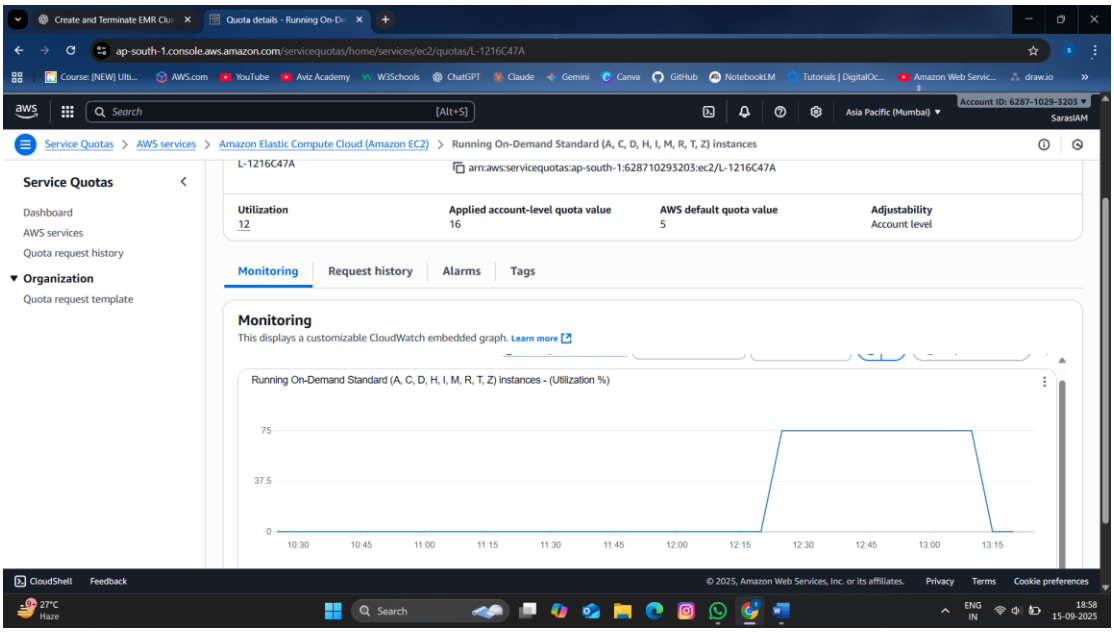
- Can’t keep the screenshot because it will take time.

Step 2: Check Service Quotas

1. Open the **AWS Console → Service Quotas**.
2. From the services list, select **Amazon Elastic Compute Cloud (Amazon EC2)**.
3. Look for the quota:
   - **"Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances"**.
4. Note the current **vCPU limit** for your account in your region (e.g., 32 vCPUs).







Result:

- You now understand the **cost implications** of running EMR clusters.
- You have identified your **vCPU quota**, which is the practical hardware limit for EMR cluster size in your account/region.

Task 4: Conceptual Understanding – Compare EMR and Databricks

Objective

The goal of this task is to develop a clear understanding of the differences between **Amazon EMR** and **Databricks**. Both are platforms for big data processing, but they serve different purposes and operate with distinct cost models and management philosophies.

Theory

- **Amazon EMR (Elastic MapReduce):** A managed Hadoop/Spark framework offered by AWS that lets you run big data workloads using different runtimes (Hadoop, Spark, Presto, Hive, etc.). It is flexible and integrates deeply with AWS infrastructure.
- **Databricks:** A unified analytics platform built around Apache Spark but extended to include ML, SQL, streaming, and governance. It abstracts much of the infrastructure management and provides advanced collaboration features.
- **Comparison Dimensions:** Key differences emerge across runtime support, pricing, primary use cases, and level of managed service.

Task: Create Comparison Table

Feature	Amazon EMR	Databricks
Supported Runtimes	Hadoop, Spark, Hive, Presto, HBase, Flink, and more (multi-runtime ecosystem).	Primarily Spark-based (but extended with SQL, MLflow, Delta Lake, Unity Catalog).
Cost Model	Pay for **EC2 instances + EBS storage + EMR software fee (per hour)**.	Pay for **DBUs (Databricks Units) + underlying cloud compute (AWS/Azure/GCP)**.
Primary Use Case	Flexible big data platform for batch, streaming, and multiple runtimes.	Spark-optimized platform for data engineering, analytics, ML, and governance.
Managed Services	AWS manages provisioning, scaling, patching, but tuning is partly manual.	Databricks manages Spark optimizations, autoscaling, notebooks, and governance tools.

Result:

You now have a clear **side-by-side comparison** of EMR and Databricks in terms of runtimes, cost model, use cases, and management. This synthesizes the insights from both Karthick's (EMR-focused) and Akhil's (Databricks-focused) sessions.

Amazon EMR vs. Databricks – Q&A Format

1. Why should I use it?

- **Amazon EMR:** Use EMR if you want flexibility to run multiple big data runtimes (Hadoop, Spark, Hive, Presto, HBase, Flink) with tight AWS integration.
- **Databricks:** Use Databricks if you want a Spark-optimized platform with built-in support for analytics, ML, and governance (Delta Lake, Unity Catalog).

---

2. When should I use it?

- **Amazon EMR:** Best when you need a **multi-runtime ecosystem**, want direct control over cluster configuration, or must integrate closely with AWS services like S3, Glue, and Redshift.
- **Databricks:** Best when teams need **collaboration, ease of use, faster productivity**, and advanced governance for end-to-end pipelines (data engineering, ML, BI).

---

3. How do I use it?

- **Amazon EMR:**
  1. Launch a cluster via AWS Console, CLI, or API.
  2. Choose runtimes (Spark, Hadoop, etc.).
  3. Submit jobs as **Steps**, via **JupyterHub**, or with **spark-submit**.
  4. Manage scaling and termination (auto-termination recommended).
- **Databricks:**
  1. Log in to Databricks workspace (AWS/Azure/GCP).
  2. Create clusters (auto-managed).
  3. Use collaborative notebooks (Python, SQL, R, Scala).
  4. Run jobs, ML pipelines, or dashboards.
  5. Manage governance with Delta Lake + Unity Catalog.

---

4. What do I gain from it?

- **Amazon EMR:**
  - Low-level control of infrastructure.
  - Multi-runtime flexibility.
  - Cost-efficient if optimized (pay for EC2 + EMR software fee).
  - Seamless AWS ecosystem integration.
- **Databricks:**
  - Higher productivity (no cluster babysitting).
  - Optimized Spark runtime → better performance.
  - Collaboration features (notebooks, workflows).
  - Enterprise governance with Delta Lake & Unity Catalog.
  - Faster time-to-value for teams.

Quick takeaway:

- Choose **EMR** if you need **control + flexibility + AWS-native integration**.
- Choose **Databricks** if you need **ease of use + collaboration + Spark optimization**.